

Serial communication protocols

Serial Communication is a method of transmitting data **one bit at a time** over a single channel or wire. It's called "serial" because data is sent **in series** — one after another — rather than in parallel (multiple bits at once).

UART Communication Protocol:

WHAT IS UART?

UART (Universal Asynchronous Receiver/Transmitter) is a way for two devices to talk to each other using just two wires:

- TX (Transmit) — sends data
- RX (Receive) — receives data

The devices send data bit by bit without a clock signal, as long as both agree on the speed (baud rate) and format.

Structure:

UART consists of the following main components:

- **Transmitter (TX)** – Sends serial data.
- **Receiver (RX)** – Receives serial data.
- **Baud Rate Generator** – Determines the speed of data transmission.
- **Shift Register** – Converts parallel data to serial and vice versa.
- **Parity Generator/Checker** – Optional, for error detection.

What You Need:

- 1x Arduino Uno
- 1x Raspberry Pi (any model with GPIO)
- Jumper wires
- USB cable for Arduino
- Common GND connection

On Raspberry Pi:

1. **Enable UART** on the Pi:
 - Open terminal: `sudo raspi-config`
 - Go to: Interface Options → Serial Port
 - **Disable login shell over serial**
 - **Enable hardware serial**
2. **Install serial tools** (if not already):

bash

Copy Edit

```
sudo apt update
sudo apt install python3-serial
```

Create a Python script to receive UART data: (#SAMPLE CODE)

python

Copy Edit

```
import serial

ser = serial.Serial("/dev/serial0", 9600) # 9600 baud rate

while True:
    if ser.in_waiting > 0:
        data = ser.readline().decode('utf-8').rstrip()
        print("Received from Arduino:", data)
```

On Arduino Uno:

1. Open Arduino IDE
2. Write a simple UART transmitter sketch:

cpp

Copy Edit

```
void setup() {
    Serial.begin(9600); // Set same baud rate as Pi
}

void loop() {
    Serial.println("Hello from Arduino!");
    delay(1000);
}
```

Testing

- After running both devices:
 - The Arduino will send "Hello from Arduino!" every second.
 - The Raspberry Pi terminal will print each line as it receives it.

Final Checklist

- TX ↔ RX wires connected crossed
- Common GND
- Voltage divider used between Arduino TX → Pi RX
- Same baud rate (e.g., 9600)

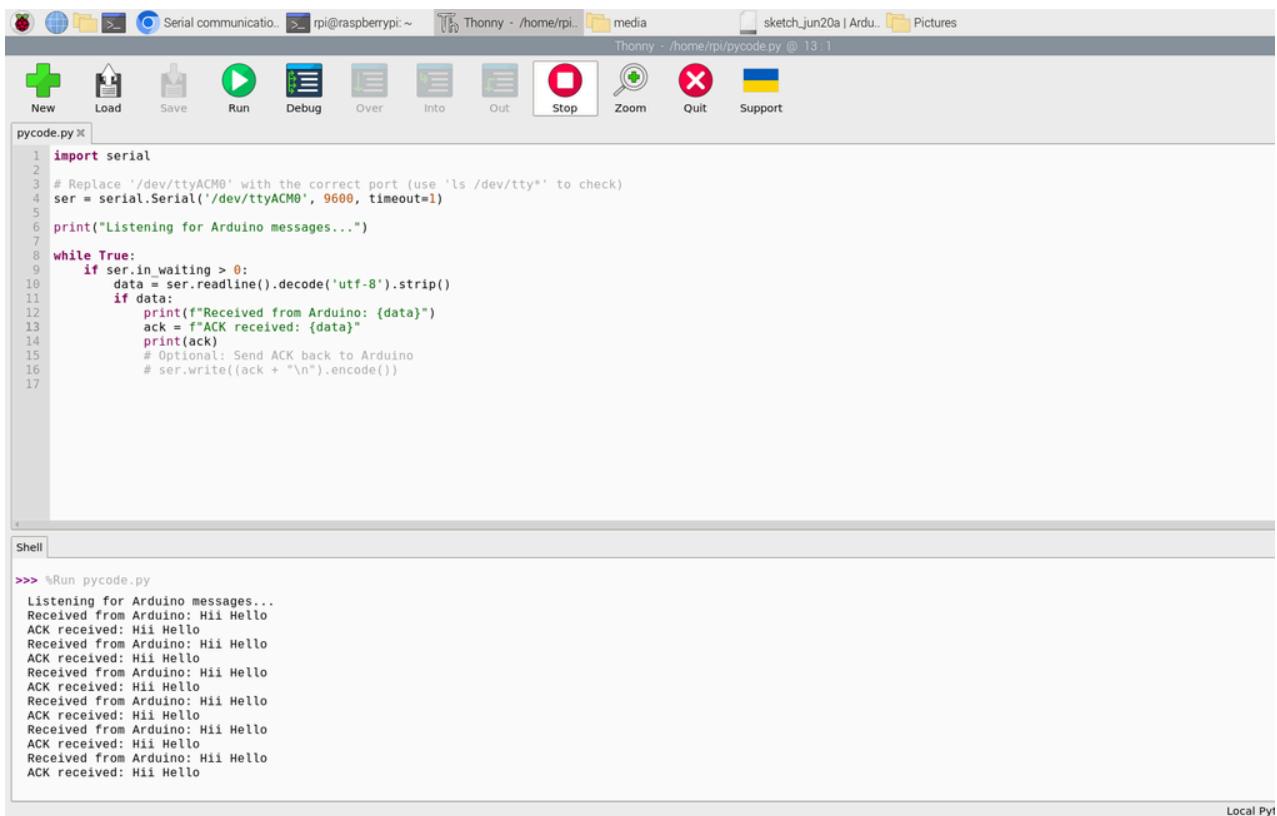
- Serial port enabled on Pi

```
sketch_jun20a
void setup() {
  Serial.begin(9600);
  delay(1000); // Give time to setup
  Serial.println("hii hello"); // Send message to RPi
}

void loop() {}
// Optional: Send repeatedly
// Optional: Wait 5 seconds
  delay(5000);
  Serial.println("hii hello");
}
```



```
/dev/ttyACM0
HiiHello
HiiHello
HiiHello
HiiHello
HiiHello
HiiHello
HiiHello
```



```
pycode.py %
1 import serial
2
3 # Replace '/dev/ttyACM0' with the correct port (use 'ls /dev/tty*' to check)
4 ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
5
6 print("Listening for Arduino messages...")
7
8 while True:
9     if ser.in_waiting > 0:
10         data = ser.readline().decode('utf-8').strip()
11         if data:
12             print(f"Received from Arduino: {data}")
13             ack = f"ACK received: {data}"
14             print(ack)
15             # Optional: Send ACK back to Arduino
16             # ser.write(ack + "\n").encode())
17
```

```
Shell
>>> %Run pycode.py
Listening for Arduino messages...
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
Received from Arduino: Hii Hello
ACK received: Hii Hello
```

SPI Communication Protocol:

What Is SPI?

SPI (Serial Peripheral Interface) is a fast, synchronous serial communication protocol. It uses a master-slave architecture — in this case, the Raspberry Pi acts as the master, and the Arduino Uno acts as the slave.

SPI uses 4 wires:

Line	Name	Direction	Description
MOSI	Master Out, Slave In	Pi → Arduino	Master sends data to slave
MISO	Master In, Slave Out	Arduino → Pi	Slave sends data to master
SCLK	Serial Clock	Pi → Arduino	Clock from master
SS or CS	Slave Select	Pi → Arduino	Selects slave device

What You Need

- 1x Raspberry Pi (any model with GPIO)
- 1x Arduino Uno
- Jumper wires
- USB cable for Arduino
- Common GND connection

SPI STEP BY STEP WIRING :

Arduino Pin	Connect to Pi GPIO	Function
11 (MOSI)	GPIO10 (Pin 19)	MOSI
12 (MISO)	GPIO9 (Pin 21)	MISO – use voltage divider
13 (SCK)	GPIO11 (Pin 23)	SCLK
10 (SS)	GPIO8 (Pin 24)	CE0 (Slave Select)
GND	GND (Pin 6)	Common ground

On Raspberry Pi (Master):

1. Enable SPI Interface:

```
bash
```

```
Copy Edit
```

```
sudo raspi-config
```

- Go to Interface Options > SPI → Enable
- Reboot if required

2 . Install SPI Python Library:

```
bash
```

```
Copy Edit
```

```
sudo apt install python3-spidev
```

Create SPI Master Code (Python):

```
python
```

```
Copy Edit
```

```
import spidev
import time

spi = spidev.SpiDev()
spi.open(0, 0) # Open bus 0, device 0 (CE0)
spi.max_speed_hz = 50000

while True:
    spi.xfer([0x01]) # Send 1 byte
    time.sleep(1)
```

Save as spi_master.py and run:

```
bash
```

```
Copy Edit
```

```
python3 spi_master.py
```

On Arduino Uno (Slave):

1. Open Arduino IDE
2. Use this SPI slave code:

cpp

Copy Edit

```
#include <SPI.h>

volatile byte receivedData;

void setup() {
    pinMode(MISO, OUTPUT); // Set MISO ready
    SPCR |= _BV(SPE);      // Enable SPI in slave mode
    SPI.attachInterrupt(); // Interrupt on SPI data received
    Serial.begin(9600);
}

ISR(SPI_STC_vect) {
    receivedData = SPDR; // SPI data register
    Serial.print("Received from Pi: ");
    Serial.println(receivedData);
    SPDR = receivedData + 1; // Optionally send something back
}

void loop() {
    // Do nothing, waits for interrupt
}
```

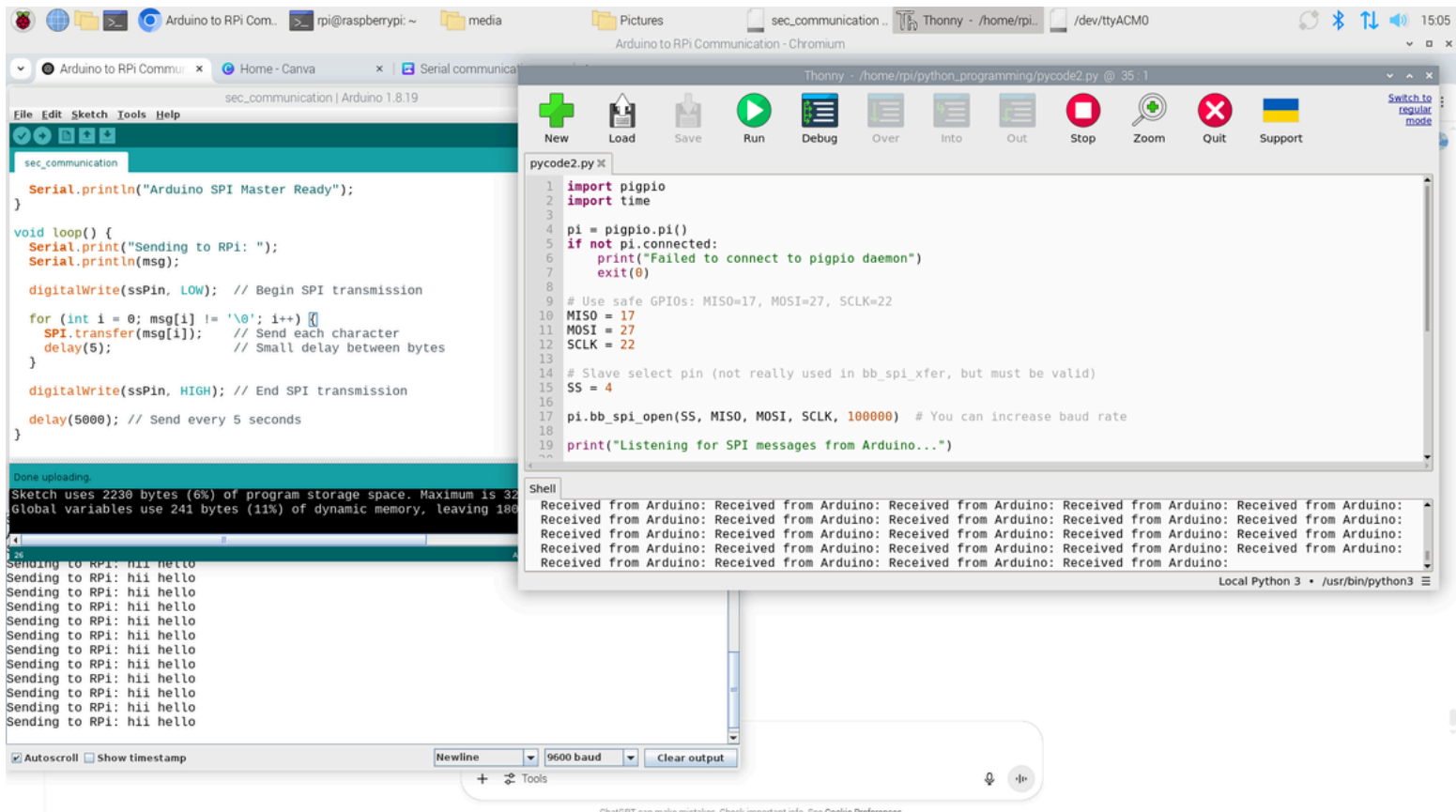
Upload this code to Arduino via USB:

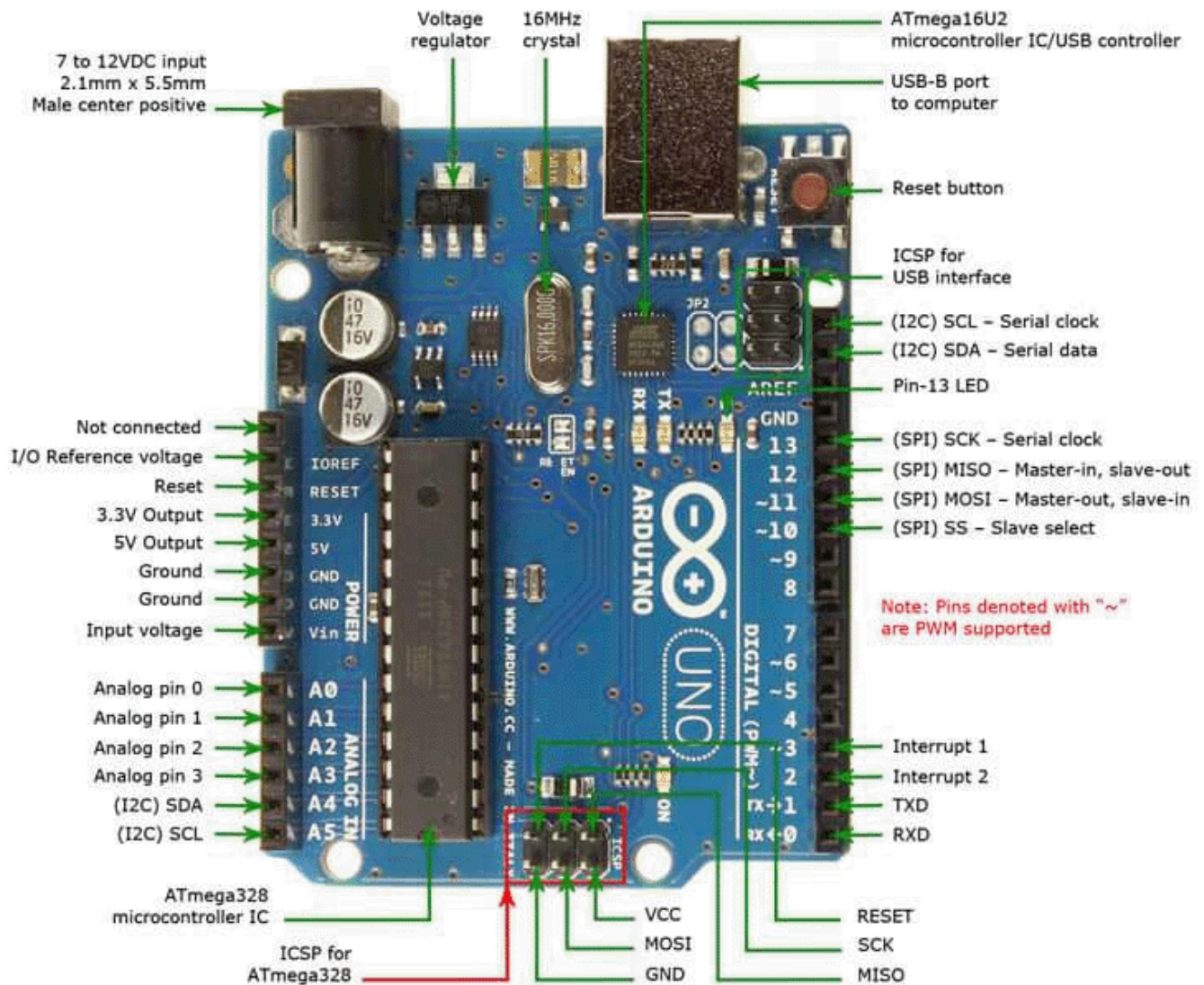
Testing

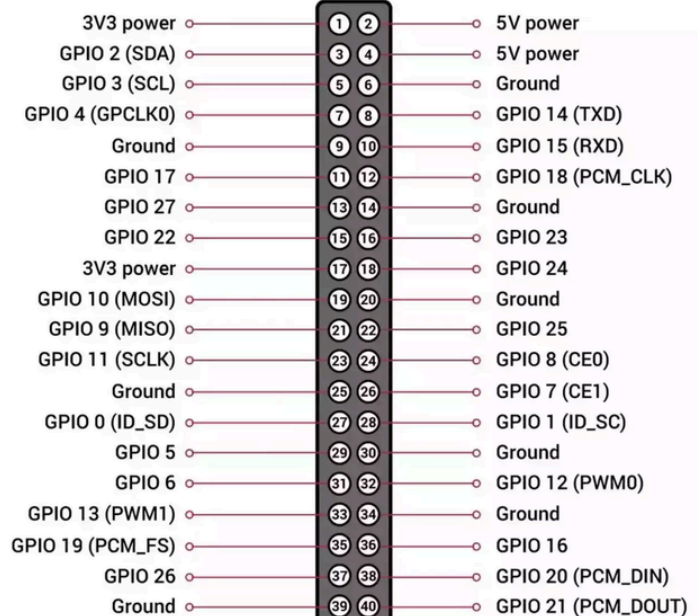
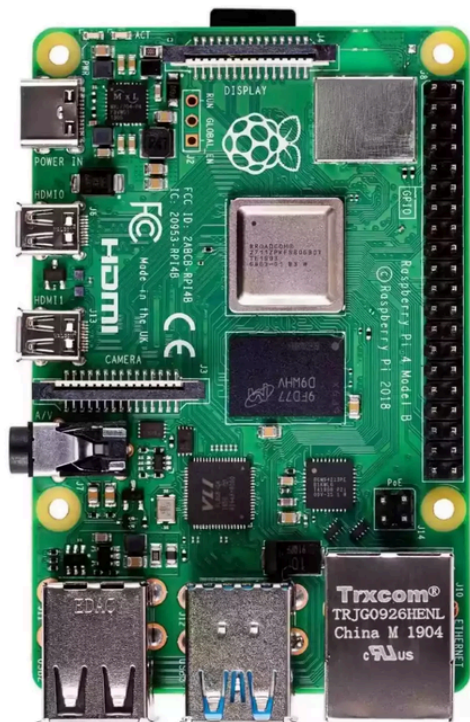
- After both scripts are running:
 - The Raspberry Pi sends byte 0x01 every second.
 - The Arduino receives it and prints the value in the Serial Monitor.
 - You can modify SPDR to send something back to the Pi.

Final Checklist

- Cross-check pin numbers
- Common GND
- SPI enabled on Pi
- MISO line uses voltage divider if needed
- Baud rate not required — SPI uses clock







40 GPIO Pins Description of Raspberry Pi 4

I2C Communication Protocol:

What Is I2C?

I²C (Inter-Integrated Circuit) is a synchronous, multi-device communication protocol designed for short-distance communication.

It supports multiple masters and slaves, but typically one master (here, the Raspberry Pi) communicates with one or more slaves (Arduino Uno in this case).

I²C Uses Two Wires:

Line	Name	Direction	Description
SDA	Serial Data	Bi-directional	Carries data to/from master and slave
SCL	Serial Clock	Master → Slave	Clock signal generated by the master

What You Need

- 1× Raspberry Pi (any model with GPIO)

- 1× Arduino Uno
- Jumper wires
- Level shifter or voltage divider (optional but recommended for 5V → 3.3V logic)
- USB cable (only for uploading code to Arduino)
- Common GND connection

I²C Wiring: Arduino Uno (Slave) ↔ Raspberry Pi (Master):

Arduino Uno Pin	Raspberry Pi GPIO	Function
A4 (SDA)	GPIO 2 (Pin 3)	SDA (Data line)
A5 (SCL)	GPIO 3 (Pin 5)	SCL (Clock line)
GND	GND (Pin 6)	Common Ground
5V	<i>Use caution!</i>	Use level shifter or separate power

On Raspberry Pi (Master)

1. Enable I²C Interface

bash

Copy

Edit

```
sudo raspi-config
```

- Go to: Interface Options → I2C → Enable
- Reboot if prompted

2. Install Required Python Libraries

bash

Copy

Edit

```
sudo apt-get install python3-smbus
```

3. I²C Master Code (Python on Pi)

python

Copy Edit

```
import smbus
import time

bus = smbus.SMBus(1) # Use I2C bus 1 (/dev/i2c-1)
address = 0x08       # Arduino I2C slave address

def read_arduino():
    try:
        data = bus.read_i2c_block_data(address, 0, 9) # Request 9 bytes
        message = ''.join([chr(byte) for byte in data])
        print("Received from Arduino:", message)
    except Exception as e:
        print("Error:", e)

while True:
    read_arduino()
    time.sleep(1)
```

On Arduino Uno (Slave)

1. Upload I²C Slave Code via Arduino IDE

cpp

Copy Edit

```
#include <Wire.h>

const char message[] = "hii hello";

void setup() {
    Wire.begin(0x08); // Slave address
    Wire.onRequest(requestEvent); // Define what to send when requested
}

void loop() {
    // Nothing needed here
}

void requestEvent() {
    Wire.write((const uint8_t*)message, sizeof(message) - 1); // Send message (omit null terminator)
}
```

Testing

After uploading the Arduino code and running the Python script on the Pi:

- Raspberry Pi will send an I²C request every second
- Arduino will respond with the message "hii hello"
- Pi will print:

csharp

 Copy  Edit

```
Received from Arduino: hii hello
```

Final Checklist

- ✓ SDA and SCL wired properly
- ✓ Common GND connection
- ✓ Pi has I2C enabled (raspi-config)
- ✓ Correct slave address (0x08)
- ✓ Level shifting if needed
- ✓ Wire.begin(0x08) on Arduino and read_i2c_block_data() on Pi